

Eksamen Webutvikling H25

Emnekode: DS3103

Emnenavn: Webutvikling

Rapport

1) Frontend-teknikker

Vi har benyttet typescript for å definere datatyper gjennom hele prosjektet. Dette har hjulpet oss til å ha oversikt og bruke riktige variabler fra interfacene. Det er laget interface for hvert objekt, og disse benyttes ved oppretting av ulike komponenter som vises i grensesnittet.

Av React hooks har vi i stor grad benyttet useContext for å kunne bruke ulike states på flere pages, som oppdateres ved endringer og vises i grensesnittet. Vi har dermed kunnet f.eks oppdatere finans ved kjøp og salg av spillere uten unødvendig mange axios-kall.

UseEffect er brukt for å hente data fra backend ved oppstart, slik at contexten ble fylt med data.

useState er også benyttet, spesielt for å gi brukere en status på handlinger. useRef har vi brukt for å koble input-elementer til en variabel som brukes for å redigere eller opprette nye objekter.

React routing er brukt for å lage routes til de ulike pagene. Vi har også laget en navbar som linker til hver side.

Axios brukes for å laste opp bilder fra bruker til backend. Det benyttes også i stor grad i context, ved å utføre CRUD-operasjoner for å hente data, legge til, oppdatere og slette objekter fra backend via service. Disse funksjonene er asynkrone og bruker await før responsen slik at datahenting kan fortsette i bakgrunnen, mens annen kode kjøres.

Vi har brukt form-element med input-felter for å la bruker legge inn verdier til et nytt objekt. Når bruker trykker på legg-til-knappen, sjekkes det om de påkrevde feltene er fylt ut. Dersom feltene er utfylt riktig, vil det opprettes et nytt objekt med verdiene, som sendes til contexten. Videre sendes objektet til service og hele veien til backend som lagrer objektet i databasen. Contexten er alltid oppdatert fra databasen, slik at grensesnittet endrer seg dynamisk.

2) Frontend og backend

Applikasjonen henger sammen ved at Service i frontend og kontrollere i backend kommuniserer med hverandre. Dette er det eneste punktet frontend og backend er koblet sammen. Med en slik arkitektur følger vi prinsippet Separation of Concerns.

I backend har vi models som holder på informasjon om ulike objekter, ved å oppfylle interfacet. Controller definerer endepunkter og gjør CRUD mot databasen gjennom DbContext til de ulike tabellene. Det benyttes try/catch for å validere om operasjonen er vellykket eller ikke. Vi har også en wwwroot mappe som inneholder startsidene for API'et og bilder som lastes opp av bruker.

Context i frontend henter data fra service og oppdaterer staten til objekter i grensesnittet hele tiden. Service sender axios-kall til controller som utfører CRUD mot dbContext i backend. Service mottar returen fra backend og sender det videre til context som oppdaterer UI. I frontend har vi interfaces som speiler interfascene i backend. Dette gjør at vi kan bruke dataene fra backend for å vise ulike komponenter i UI.

For at frontend skal få tilgang til API'et er CORS konfigurert i backend. Vi har konfigurert backend slik at alle kan gjøre kall mot API'et. I en reell situasjon ville vi begrenset hvem som kan gjøre kall her.

Når en bruker laster opp et bilde, vil dette sendes til ImageUploadService i frontend. Videre sendes det til ImageController i backend og lagres i images-mappen i wwwroot. Image-navnet lagres i objektet og vises i frontend ved å bruke filsti til wwwroot i tillegg til bildenavnet. Vi har ikke implementert generering av tilfeldige bildenavn, noe som kan skape konflikt hvis brukere laster opp bilder med navn som finnes i databasen.

Når service sender et axios-kall til controller, forventer den en respons tilbake. Responsen er en melding om det gikk bra eller ikke og eventuell data. Dette skjer ved at vi i controller returnerer Ok() med statuskode eller ønsket data i try. I catch returneres StatusCode(500) hvis noe har gått galt på server siden.

Når et athlete objekt blir opprettet i frontend tas det i mot av service som sender en [axios.post](#) forespørsel til controller. Controller tar objektet i HttpPost funksjonen. I funksjonen sjekkes det om objektet har data, og hvis det har data blir det lagt til i databasen. Hvis lagringen er vellykket returneres Id-en som blir tildelt i databasen, i tillegg til objektets data. Objektet hentes av context via

service, og det opprettes et nytt objekt med id som er tildelt i databasen, slik at dette kan vises i grensesnittet uten å hente alt fra databasen på nytt.

3) Universell utforming

Vi har hatt fokus på universell utforming gjennom utvikling av løsningen, slik at brukere med forskjellige forutsetninger kan få verdi av produktet. Vi har vært bevisst på å plassere komponenter slik at man intuitivt skjønner hva de skal brukes til. Vi har hatt fokus på eksternt design, ved å designe nettsiden på en kjent måte, som ligner på nettsider som bruker har vært innom fra før. Farger på knapper er også valgt ut ifra eksternt design, ved at grønn knapp gjør noe positivt og rød knapp noe negativt. Dette gir brukeren en tydelig indikasjon på hva en knapp gjør.

Alle bilder har alt-tekst, slik at dersom bildet ikke kan vises blir det representert i form av en beskrivelse. Alt-teksten er også viktig for svaksynte som bruker opplesningsverktøy. Vi har sørget for å ha gode kontraster som oppfyller WCAG slik at tekst blir lett å oppfatte for brukere med ulike funksjonsnedsettelse.

Vi har også lagt vekt på tydelige tilbakemeldinger til bruker ved handlinger. Det er både i form av tekst som forklarer hva som har skjedd, og instruksjoner ved feil utfylling. Dersom bruker skriver inn noe feil, vil ikke handlingen gå gjennom. Dette hindrer at bruker kan legge til feil objekter. Ved å legge til nye objekter dukker de opp på siden umiddelbart og gir et tydelig tegn på at handlingen var vellykket. Det samme skjer når en bruker sletter et objekt ved at det forsvinner fra siden.

For å gjøre siden tilgjengelig ved ulike skjermstørrelser, har vi laget et responsivt design ved å ta i bruk grid og flex-box. Dette sikrer at man kan bruke løsningen på ulike enheter. På mobil vises elementene i en kolonne, som gir en god oversikt og størrelse på elementer. På nettbrett øker bredden til to kolonner, slik utnyttes plassen bedre uten å påvirke synligheten. Ved større skjermstørrelser, som pc eller monitor, vises det tre og fire elementer i bredden.

Knapper er plassert nederst i tilhørende elementer for å gjøre det lettere for bruker å nå på mobil og nettbrett. Dette er også gjort for at brukeren skal få med seg innholdet før de utfører en operasjon.